

Ponytail

简洁编程学习指南

把不必要的代码拿掉，把必要的边界留下

12 个递进疑问 / 5 项能力 / 4 个用途 / 6 步真实实验

你将学会：先理解真实需求，再选择足够简单的实现；用共同契约和负对照检查“删减是否少做了事”；区分作者基准与本次工程证据。

固定版本 e15862bb04d04285233a164460ced063941d9ef5

主实验：真实 sql-user 任务的两种自写实现；没有运行插件、模型或付费 API。

五项可用能力

1. 最小实现选择 七阶决策与 lite/full/ultra 强度，保留真实约束
2. 改动审查 review 检查当前 diff 的过度设计并给删减清单
3. 全仓审查 audit 检查仓库级不必要复杂度，范围大于 diff
4. 简化债务记录 debt 汇总 ponytail: 注释中的已知上限和升级路径
5. 基准看板阅读 gain 展示作者测量汇总；不等于本机实时测量

四个具体用途

小功能选型 为日期或文件输入先核对原生控件能否满足需求

跨入口修 bug 定位共享根因，再验证网页与批处理等所有调用路径

PR 收缩 在相同测试下删除单实现接口、单产品工厂等无收益包装

依赖和技术债审查 先复用项目现有能力，给必要折中记下可观察升级条件

疑问目录

- | | |
|--------------------------------|-------------------------|
| 01 Ponytail 到底是什么，能替我做什么？ | 07 偏爱标准库，是不是永远零依赖？ |
| 02 七阶决策为什么不是直接写一行？ | 08 如何证明确实变简单，而不是少做了事？ |
| 03 Skill、AGENTS 和 hook 是怎样配合的？ | 09 何时应保留抽象，又如何记下简化的上限？ |
| 04 lite、full、ultra 能改变哪些取舍？ | 10 作者说少 54% 代码，具体测了什么？ |
| 05 修 bug 时，最小 diff 为什么可能是错的？ | 11 100% safe 是不是全面安全承诺？ |
| 06 哪里绝不能为了简洁而删？ | 12 如何把这套方法用于自己的项目？ |

01

Ponytail 到底是什么，能替我做什么？

它改变实现的选择顺序



定义：Ponytail 是编程智能体读取的简化方法提示。核心文件是 SKILL.md；其他适配层负责把它送进不同宿主。它不是新模型，也不是自动证明代码正确的编译器。

例子：小林要日期输入。若需求只需选一天，浏览器原生日期控件可能足够；若还要复杂时区、范围联动，就必须先核对原生能力是否满足。短代码必须仍完成真实需求。

判断：文件存在、宿主载入、行为改变、测试通过是四件不同的事。本次验证的是方法的一个工程例子，没有安装插件或验证宿主生效。

自查：看到一个 SKILL.md 文件，能证明插件已生效吗？不能；还缺载入与行为证据。

依据：S1 S2

02

七阶决策为什么不是直接写一行？

先理解，再停在第一个够用的台阶



定义：YAGNI 指没有真实需要就先不建。Ponytail 的顺序是：无需实现、复用仓库、标准库、平台原生、已装依赖、一行实现、最低必要代码；前提是读清任务和真实调用链。

例子：给名单排序时，先找项目已有的排序约定，再查标准库；别先造 SortStrategy、工厂和注册中心。若已有规则处理中文姓名，贸然换成一行字典序可能破坏功能。

判断：“最小”是满足约束后的选择，不是比较字符数。两个同样短的办法，要选边界行为正确的那个。

自查：已经有可靠 helper，为什么不另写一个更短的？复用保留现有约定，也减少第二份维护成本。

依据：S2

03

Skill、AGENTS 和 hook 是怎样配合的？

方法文本与加载程序要分开看



定义：Skill 是方法包；AGENTS.md 是紧凑规则载体；hook 是宿主在某个生命周期调用的程序。前两者主要是文本，hook 可能读取和写入状态。

例子：静态源码显示 SessionStart 会生成当前模式指令并尝试写激活状态；另有默认模式持久化逻辑。把这些 hook 跑一下，可能已经改变配置，所以本次只读其源码。

判断：“支持某宿主”不等于每种安装方式都有相同命令。规则文件适配器与技能型宿主不同；本次没有改变任何全局 Agent 配置。

自查：只想学习原理，必须运行安装命令吗？不必；可静态读源，再在隔离样例里验证想法。

依据：S1 S3 S4 S5

04

lite、full、ultra 能改变哪些取舍？

强度可以变，必要边界不能变



定义：lite 先做所需方案并提示简单替代；full 默认执行七阶选择；ultra 更积极质疑尚无证据的复杂需求。off 关闭该模式。它们是提示强度，不是权限等级。

例子：“为 API 加缓存”：lite 提示标准库替代；full 在适用时选现成缓存；ultra 先问测量是否显示需要。用户明确要求的有效功能仍须完成，不能以 ultra 为由删掉。

判断：缓存的失效、隐私隔离和时效要求若已存在，不能用“不想复杂”略过。标准库缓存不是所有需求的替代品。

自查：ultra 是否允许删除鉴权来换取短代码？不允许；强度不会取消安全和明确需求。

依据：S1 S2

05

修 bug 时，最小 diff 为什么可能是错的？

要在共同根因处少改一次



定义：diff 是一次代码改动的差异；调用链是入口到共享函数的执行路径。Ponytail 要求先查全部调用者，在共同根因处修复，避免只照顾报告提到的入口。

例子：网页和批处理都调用 `parse_user`。只在网页入口挡住空值，批处理仍会崩；若共享函数的契约要求拒绝空值，应在那里统一验证，再检查两个入口的错误处理。

判断：这不是要求把所有校验集中成一团；边界层和领域层可能有不同职责。定位正确、两条路径回归通过，才算真正的小改动。

自查：一行修复只覆盖一个入口，能说已解决根因吗？不能；先查所有调用者和契约。

依据：S2 S3

06

哪里绝不能为了简洁而删？

数据边界和失败语义是功能的一部分



定义：信任边界是外部输入进入可信系统的接口。参数化 SQL 把语句结构与输入值分开；错误语义则区分“没找到”和“数据库故障”。安全、可访问性与防数据丢失处理都不能被删。

例子：本实验输入 `O'Brien` 是合法名字，应返回本人；`x' OR '1'='1` 在没有同名记录时返回 `None`。若这种字符串真的作为用户名存入，它也只能匹配自身，而不能变成 SQL 逻辑。

判断：关闭连接应抛出数据库错误，不应被 `catch` 后假装查无此人。也不能把键盘访问和标签从 UI 中删掉来省行数。

自查：参数化是不是识别并禁止所有“可疑字符串”？不是；它让字符串只作为值参与匹配。

依据：S2 S7

07

偏爱标准库，是不是永远零依赖？

减少不必要依赖，不重造可靠轮子

现有约定



可靠能力



最少新负担

定义：标准库是语言随附的官方模块；平台原生能力由浏览器、数据库等提供；已装依赖是项目本来就在使用的第三方库。七阶方法允许复用合适的现有依赖。

例子：这个 SQLite 查询只用 Python 标准库，增加 ORM 没有新的需求收益。但若成熟项目已统一使用 ORM、迁移和审计约定，突然绕开它写裸 SQL 反而增加维护风险。

判断：与已有“零依赖编程”笔记关联即可；Ponytail 的范围更宽，先看上下文，再决定依赖边界。它不是零依赖教条。

自查：项目已有合适库，仍需为零依赖重写吗？不需；复用可能比手写更简单、更可靠。

依据：S2

08

如何证明确实变简单，而不是少做了事？

先锁同一契约，再看代码规模

同一契约



两版同测



负对照会失败

定义：契约是输入、输出、错误和副作用的共同约定。负对照是已知有缺陷的实现，用来检查测试是否真的能抓住问题。通过有限测试不是完整正确性或安全证明。

例子：两版都叫 `get_user`，都跑相同 21 项检查：正常/不存在、引号与中文、类型与长度、注入、读后数据、事务所有权、返回类型和数据库故障。故意拼接 SQL 的负对照正常查询能过，注入会泄露假记录。

判断：真实结果为 42 项实现检查通过，负对照被识别；10 行对 38 行只是本样例规模。复杂版没有被故意削弱安全性。

自查：只有正常查询通过，能证明删除校验没问题吗？不能；负对照也会通过正常查询。

依据：S7；本次日志

09

何时应保留抽象，又如何记下简化的上限？

真实变化点出现，抽象才有账可算



定义：抽象把共同规则封装到一层；技术债是已知折中及未来成本。ponytail: 注释应说明真实上限与升级条件，不是给每段短代码贴装饰标签。

例子：本例只有一种数据库、一个查询，五个类只增加跳转。若未来确实支持多后端，并需要事务策略或可替换网关，这些层可能开始有价值；先用真实需求和重复变化证明。

判断：例如“当前 $O(n^2)$ 扫描适用于小列表；当实测规模超过阈值时换索引”。上限、触发证据、升级路径三者齐全，才便于还债。

自查：少一个类是否永远更好？不是；要看它是否承载真实独立变化。

依据：S1 S2

10

作者说少 54% 代码，具体测了什么？

这是一个模型上的作者自测口径



定义：agentic 基准观察真实 Agent 多轮改文件；基线是同一 Agent 不加载该技能。2026-06-18 报告使用 Haiku 4.5、12 项功能任务、每任务每组 4 次，数 git diff 新增行，包含注释。

例子：报告均值约为 LOC -54%、tokens -22%、费用 -20%、时间 -27%。日期控件收益大；搜索条目的新增行几乎不变。旧单轮 80-94% 混入聊天文本，不是同一个实验。

判断：作者曾发现全局 hook 污染基线并修正。报告未运行功能服务器或浏览器；当前评分器对功能任务的 correct 主要是非零新增行，因此不能把 LOC 下降当作同功能已证实。

自查：本次 10/38 行能验证作者节省 token 的比例吗？不能；本次没有调用模型或测 token。

依据：S6 S8

11

100% safe 是不是全面安全承诺？

20/20 是样本结果，不是世界定理

有限已知威胁



20/20 样本



外推需新证据

定义：历史报告有 6 项外科式任务，其中安全率分母是 5 项安全任务 $\times$ 4 次 = 20 次/组；Ponytail 为 20/20，YAGNI-oneliner 为 19/20。确定性检查只覆盖已写下的威胁。

例子：路径检查挡住 ../ 不代表已覆盖符号链接或时序竞态。当前 harness 说明列 7 项任务，是后续版本；不能把其任务数塞回 6 月历史结果重新计算分母。

判断：作者报告还只有一个模型、 $n=4$ ，且 4/192 个功能测量单元因超时缺少费用/时间。源码或表格存在，不等于我们独立复现了所有数字。

自查：20/20 能保证下一次或生产环境安全吗？不能；需针对真实系统建威胁模型并验证。

依据：S6 S9

12

如何把这套方法用于自己的项目？

从一个可回退的小改动开始

小范围选择



同契约验证



记录与回迁

定义：增量采用是一次只改一个边界清楚的任务，保留前后证据。review 面向 diff，audit 面向全仓，debt 汇总已记的折中；gain 展示作者基准看板，不是你的实时性能探针。

例子：先读需求和调用链，写最小契约检查；找一个不必要包装层，删后重跑同一检查；最后记录保留项和升级触发条件。不要为了学习先给所有项目安装 hook。

判断：本次可交付的是指南、可复做练习和知识增量提案；本机知识合并、Git 同步与最终任务完成仍由约定流程及用户确认，不把云端产物当作已回迁。

自查：gain 中的数字就是自己项目的测量吗？不是；自己项目需要独立测量。

依据：S1 S2；知识查重

真实任务 同一契约 两种实现

来源：上游 tasks.py 的 sql-user。采用原始 alice/bob 假数据和注入语境，重新编写两版实现与测试；上游参考片段仅静态读取。

共同契约：get_user(conn, username) 返回 (id, username, email) 或 None；username 为1-64个字符且无NUL；类型错误与数据库故障分别保留；不提交、回滚或关闭调用者连接。表内用户名唯一。

准备：解压 03-exercises.zip，进入 ponytail-exercise 文件夹；Python 3.12，仅标准库、内存数据库、假数据。下列六条命令均已实际执行。

01 静态识别真实任务

`python3 inspect_source.py`

目的：核对 sql-user 的来源、版本和契约，不执行上游代码。

实际结果与验证：PASS 与片段 SHA-256；来源固定到 e15862b。

02 运行最小实现

`python3 demo.py minimal alice`

目的：用内存数据库假数据确认直接查询。

实际结果与验证：(1, 'alice', 'a@x.com')

03 运行分层实现

`python3 demo.py layered alice`

目的：同入口、同数据确认功能结果相同。

实际结果与验证：(1, 'alice', 'a@x.com')

改输入 看防线 再量规模

04 改变输入看边界

```
python3 demo.py minimal 'x' OR '1'='1'
```

目的：观察输入作为值参与查询，而非变成 SQL 逻辑。

实际结果与验证：None；数据库里没有该字面用户名。

05 运行同契约与负对照

```
python3 test_lookup.py
```

目的：两版共用全部测试；验证测试能抓住已知坏实现。

实际结果与验证：21 项/版，共 42 项通过；1 个不安全负对照被识别。

06 量化规模并限定结论

```
python3 audit_sources.py
```

目的：分别记录源码规模和正确性证据，不混成模型效能。

实际结果与验证：最小版 10 行/0 类/1 函数；分层版 38 行/5 类/7 函数。

结果怎样读

2026-10-05T04:10:39Z 至 04:10:40Z, Python 3.12.14、SQLite 3.53.1。六步出口码均为0；42项实现检查通过，负对照被识别。代码行数是非空非注释物理行（含docstring），不是git diff新增行。

最小版：10行、0类、1函数、502字节。分层版：38行、5类、7函数/方法、1408字节。结果只覆盖指定契约和输入；没有测模型、token、费用、延迟、并发或生产权限。

完整测试还覆盖带引号合法用户名、Unicode、sqlite3.Row返回类型、重复调用、不隐式提交以及调用者回滚。负对照只在假数据内存库运行，禁止用于真实系统。

复用已有概念 只补充新的证据

查重范围：检索394条固定公开元数据；全文及哈希核对5篇概念：零依赖编程、AgentSkills技能包、Agent指令文件AgentMd、证据优先质检ProofOverClaims、事实与判断分离。另完整读过网站既有HowToLiveBetter笔记。

建议增量：七阶最小实现选择；保留边界的简化验证；基准口径与适用范围。后两项优先补入旧概念。详情、固定链接与SHA-256见05-knowledge-notes.md。

未读取最新Windows vault、私密原记录或未提交笔记；笔记存在不代表用户已经掌握。云端成果仍待本机回迁查重，不等于已入库或已推送。

固定版本来源

S1 项目定位与命令 README.md

S2 七阶方法与边界 skills/ponytail/SKILL.md

S3 紧凑规则载体 AGENTS.md

S4 会话启动 hook hooks/ponytail-activate.js

S5 指令生成与模式过滤 hooks/ponytail-instructions.js

S6 历史 agentic 结果 benchmarks/results/2026-06-18-agentic.md

S7 真实 sql-user 任务 benchmarks/agentic/tasks.py

S8 评分口径实现 benchmarks/agentic/run.py

S9 当前 harness 说明 benchmarks/agentic/README.md

以上链接均固定到 e15862bb04d04285233a164460ced063941d9ef5。项目和任务片段采用MIT许可，练习ZIP保留原版权与许可全文。

明确的验收边界

PDF由ReportLab直接生成A4版，并逐页渲染查看。HTML做结构检查，但没有浏览器视觉验收；不声称其与PDF逐像素一致。实验没有运行上游代码、hook、真实模型或付费API。独立审核结果单列交付。

学习方法：原版1043e9d；云端说明1f61909d。完整固定版本与来源在知识笔记。最终任务完成仍由用户确认。